

Universidad Nacional Autónoma de México

Facultad de Ingeniería



Sistemas Distribuidos

Semestre 2027 - 1

Laboratorio S9. **Mensajería con RabbitMQ**

Alumno: Fernández Herrera Mauricio

Equipo: Pumaverso Distribuido.

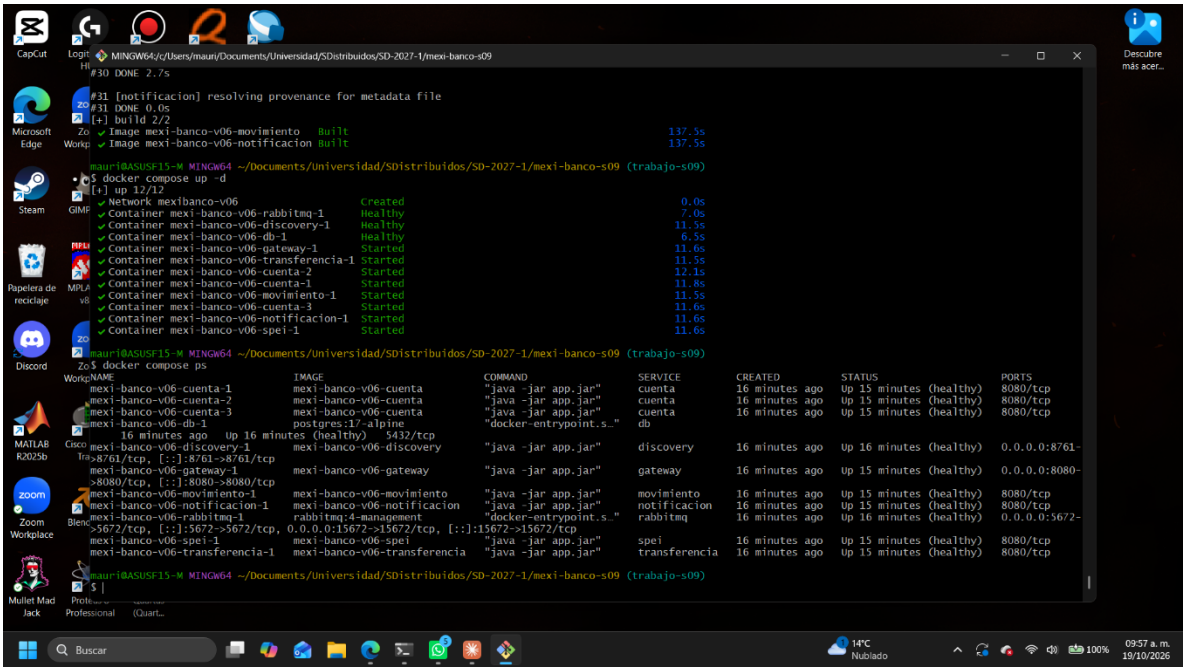
Profesor: ING. OSCAR MANUEL RUIZ HURTADO

Grupo: 02

Entrega: 23/10/2026

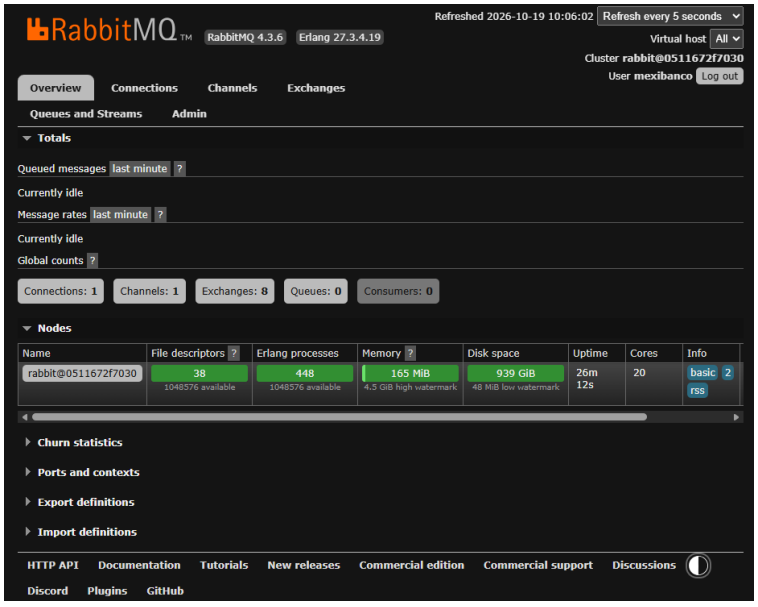
1.- Antes de clase, en tu casa. 19/10/2026. 9:58am

Después de descargar la rama base del laboratorio y crear nuestra rama de trabajo individual, construimos y levantamos. Esperamos a que todos se encuentren en estado healthy.



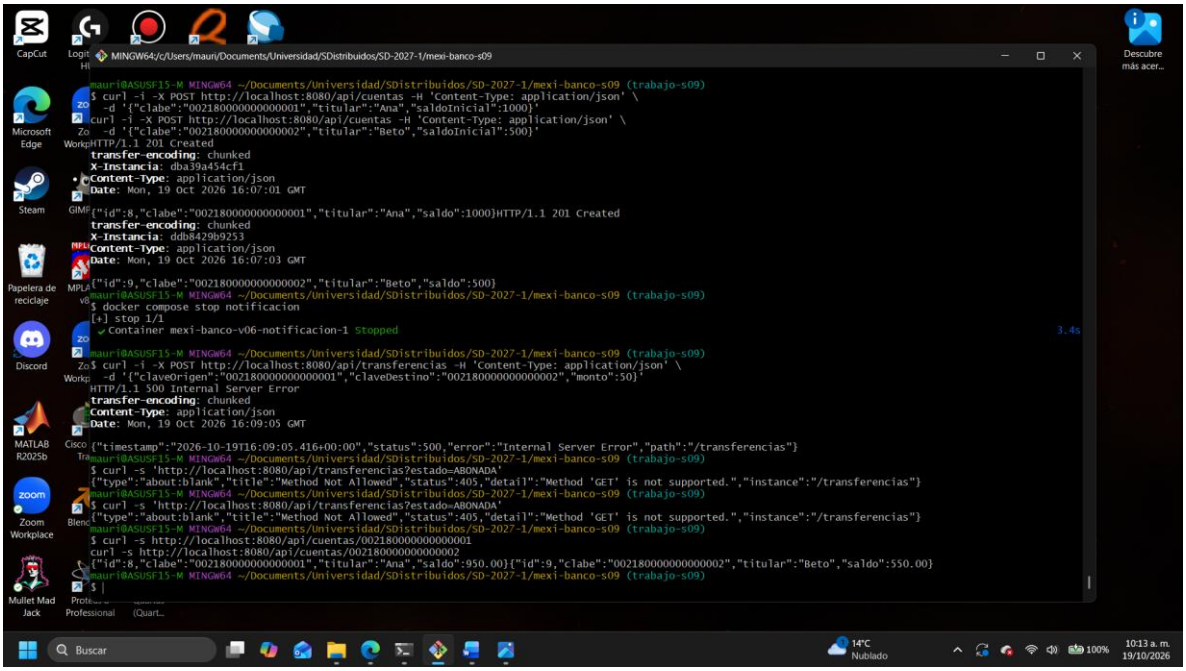
2.- Levantar la base y ver el problema. 19/10/2026. 10:03am

Abrimos el panel de RabbitMQ.



Creamos las cuentas de Ana y Beto con saldos de 1000 y 500 respectivamente.

Detenemos notificación y transferimos 50.



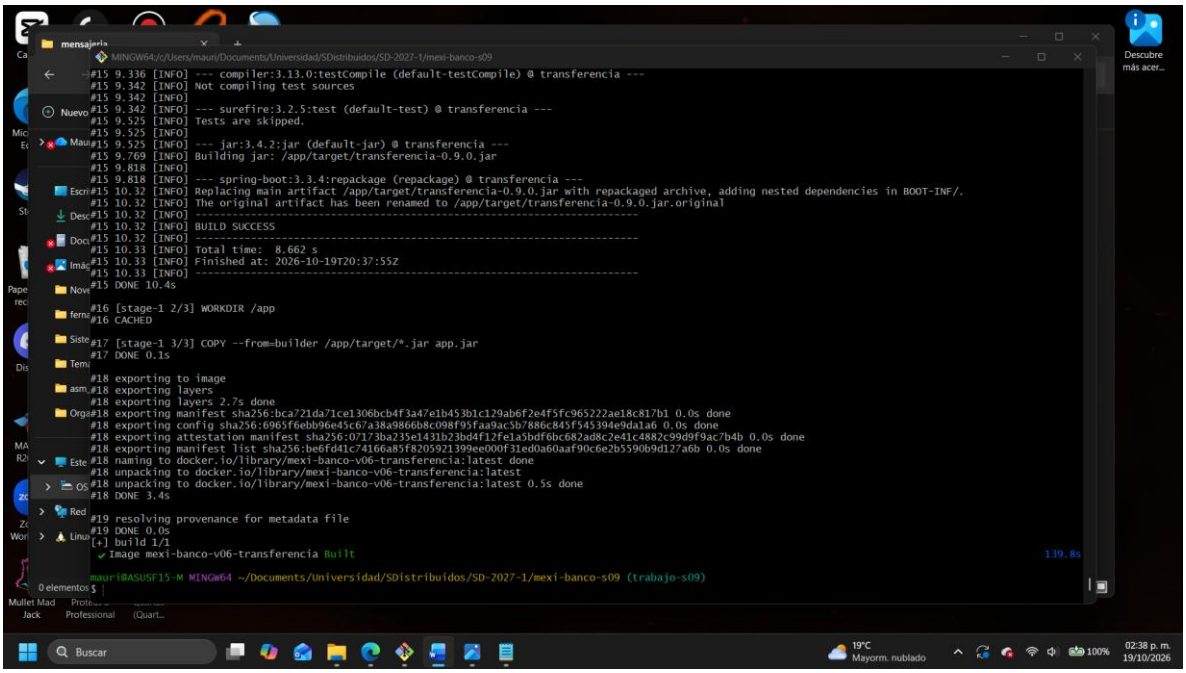
Observamos que la transferencia se hizo, aunque respondiera con error 503.

Podemos decir que el cliente recibe un error porque la saga todavía esperaba la respuesta síncrona del servicio de notificaciones por HTTP y, al fallar este servicio, la operación devuelve un error, aunque el cargo, el abono y los asientos ya se hayan realizado.

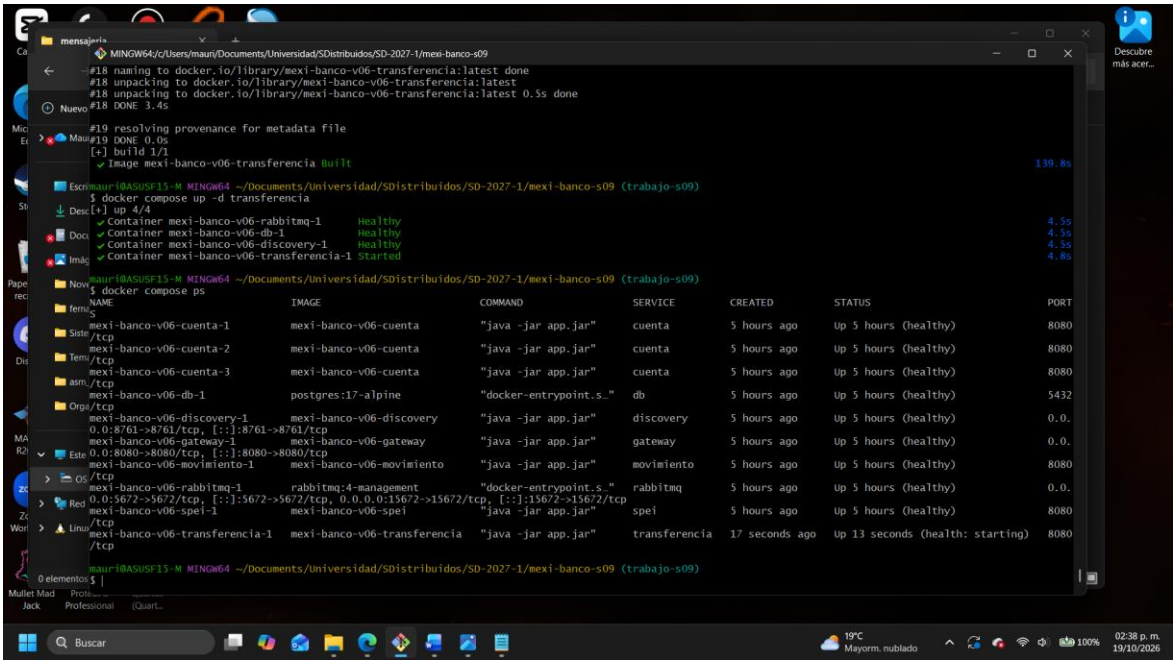
3.- Publicar el evento. 19/10/2026. 2:36am

3.1.- Que cada transferencia nazca con su eventoId

Modificamos Transferencia y volvemos a construir para que se apliquen los cambios realizados.

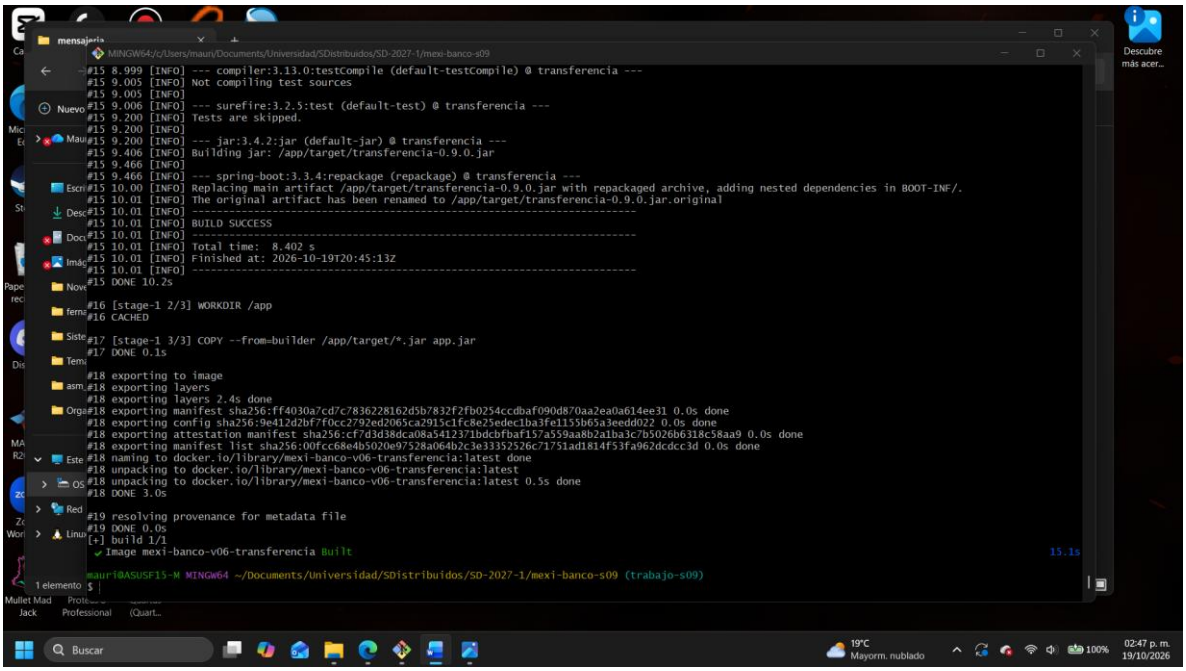


Esperamos a que esta se encuentre en estado Healthy.



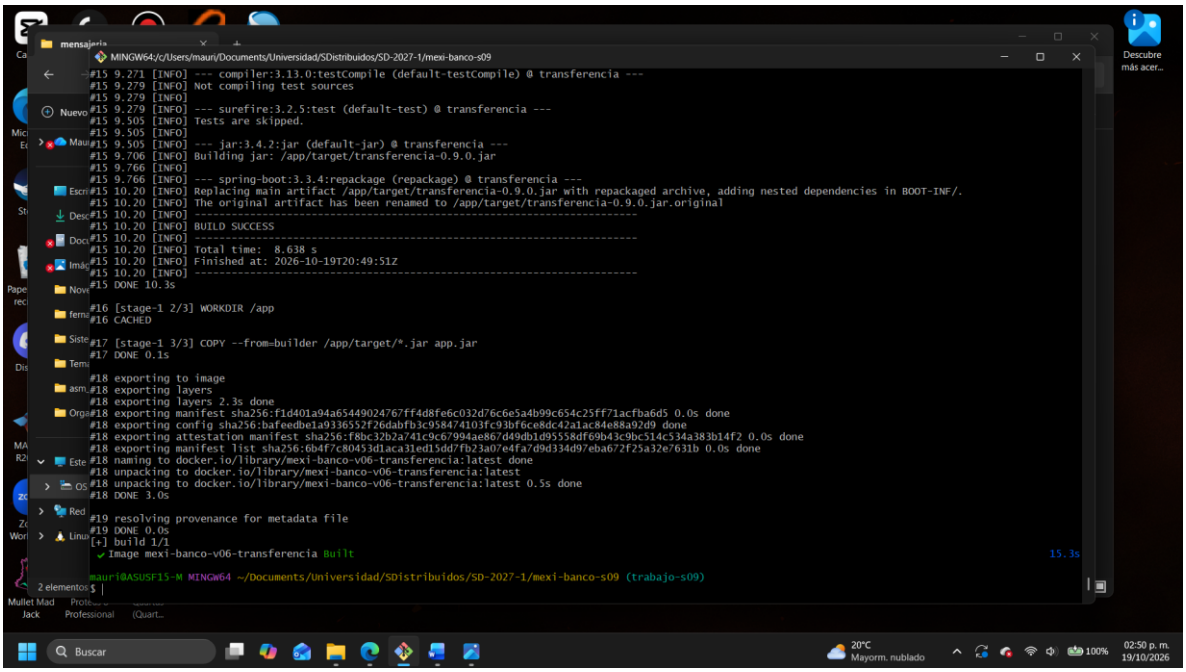
3.2.- El evento

Tras haber creado el nuevo directorio de mensajería y el archivo EventoTransferencia.java, volvemos a construir transferencia.



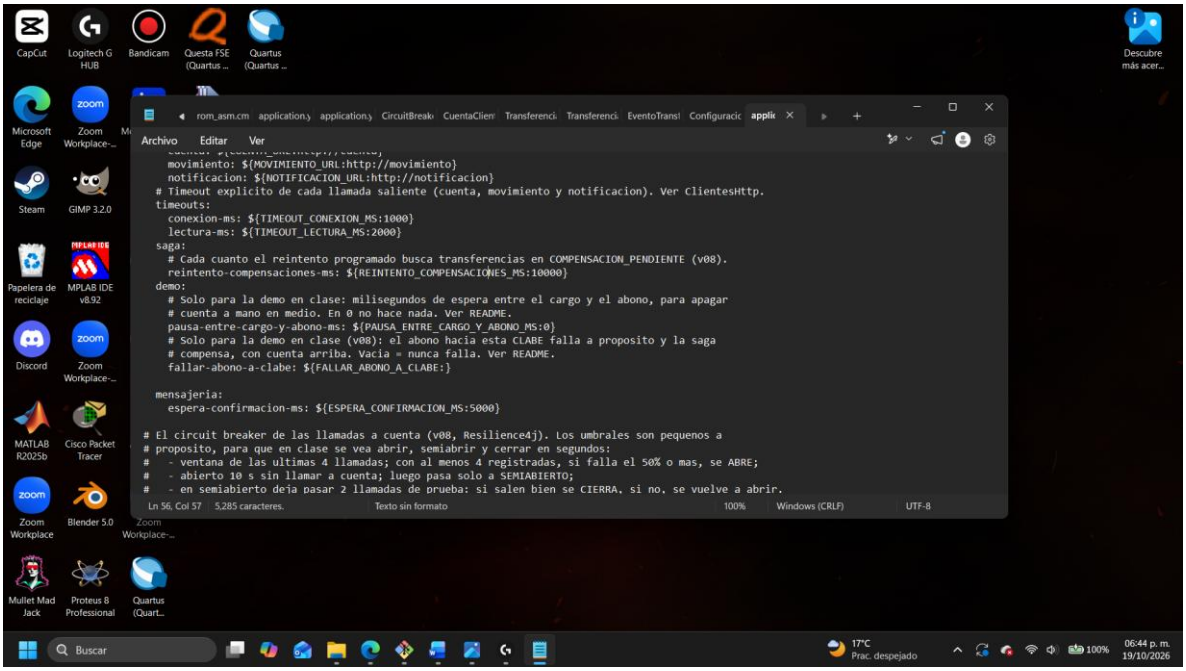
3.3.- El exchange y el convertidor JSON

Creamos mensajería/ConfiguracionMensajeria.java y volvemos a construir transferencia.



3.4.- Que el broker confirme: publisher confirms

Hacemos los camios necesarios a RabbitMQ y la espera de la confirmación en la clase mexibanco.

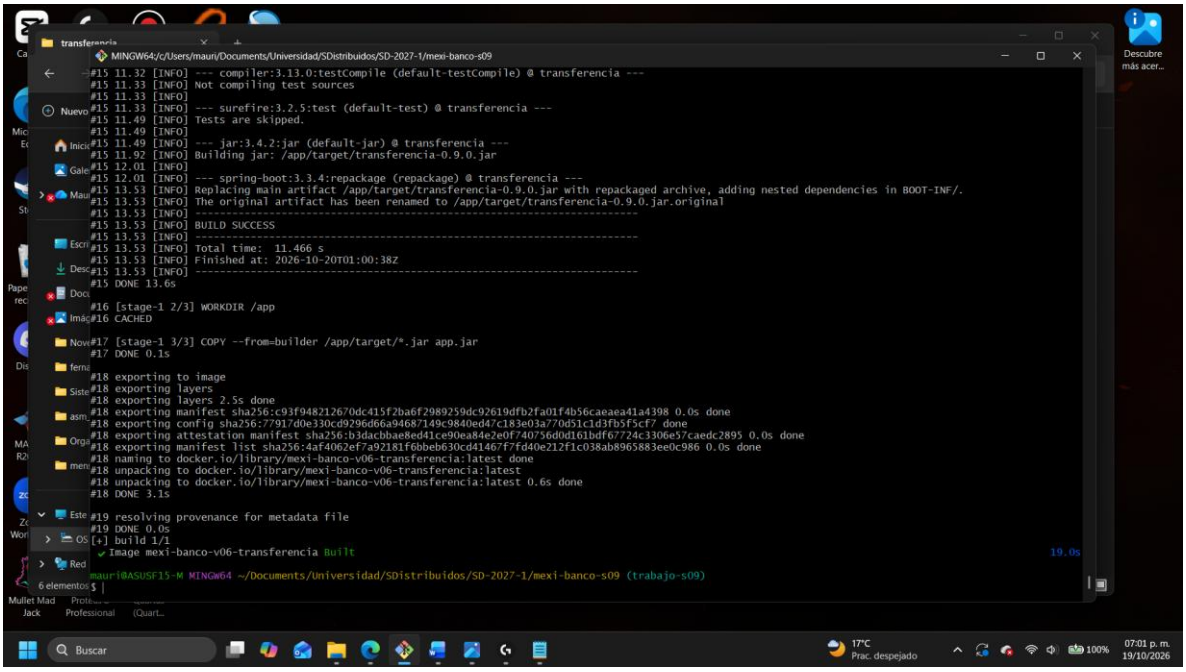


3.5.- El publicador

Creamos los archivos EventoNoPublicadoException.java y PublicadorDeEventos.java. que se encargarán de enviar el evento a transferencias con su routing key y marcar el mensaje como persistente e incluir su ID. La configuración de RabbitMQ que agregamos en el paso 3.4 ya define el tiempo de espera y habilita mandatory, los publisher confirms y los mensajes persistentes.

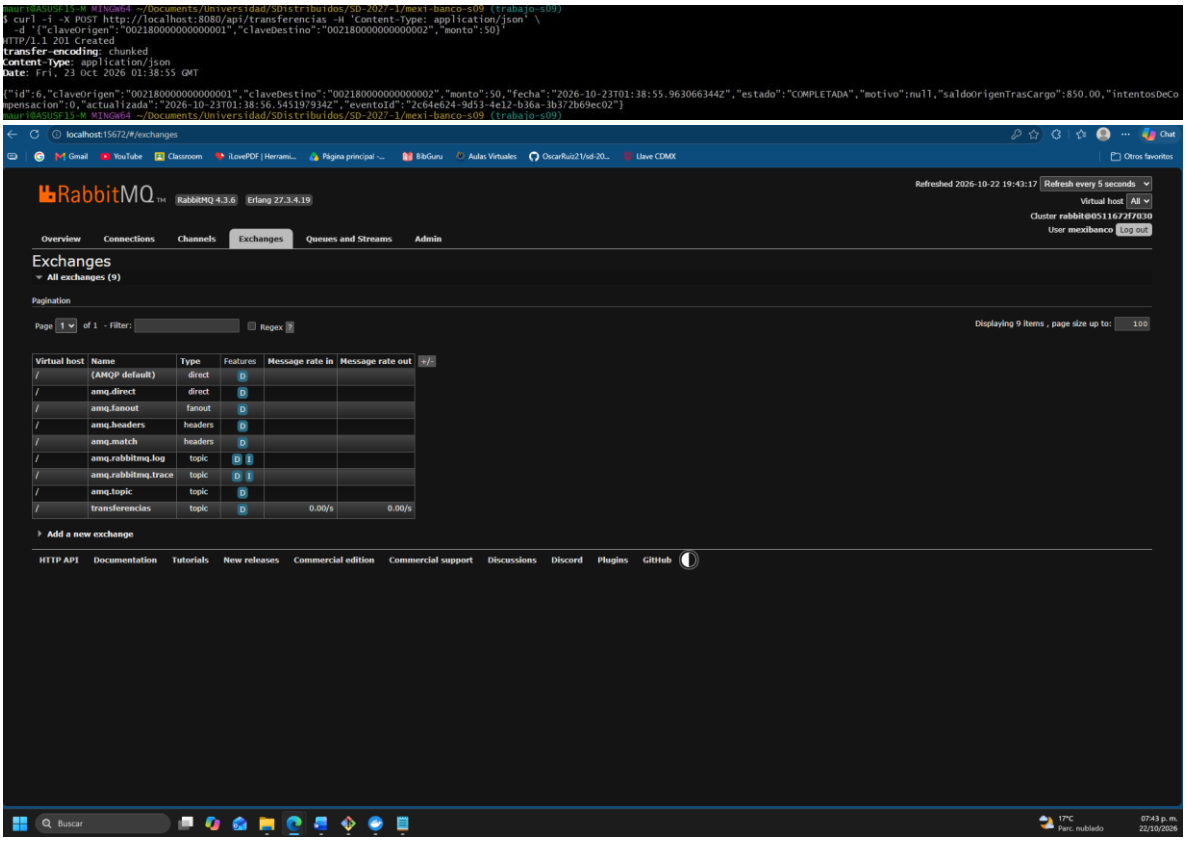
3.6.- La saga publica en lugar de llama

Habiendo hecho todos los cambios necesarios en transferencia.java, volvemos a construir la imagen de transferencia para verificar que no haya ningún problema.



3.7.- Pruébalo con notificación apagado

Tras apagar notificación, consultamos el estado de la transferencia tanto en gitbash como en RabbitMQ.



Los publisher confirms permiten confirmar que RabbitMQ recibió y aceptó la publicación del mensaje. Sin embargo, no garantizan que el consumidor lo haya recibido, procesado o guardado en la base de datos.

Si el servicio de notificaciones está apagado y no existe ninguna cola enlazada al exchange con la routing key correspondiente, RabbitMQ puede confirmar la publicación, pero, como está activada la opción mandatory, el mensaje se devuelve al publicador al no encontrar una cola que lo reciba. En este caso, el publicador detecta la devolución y registra el error, aunque la transferencia permanece en estado completada.

Por otro lado, si el proceso de transferencia muere justo después de ejecutar guardar(t) y antes de avisar, la transferencia puede quedar registrada como completada en la base de datos sin que el evento llegue a publicarse. Los publisher confirms no evitan este problema porque todavía no se ha enviado ningún mensaje a RabbitMQ.

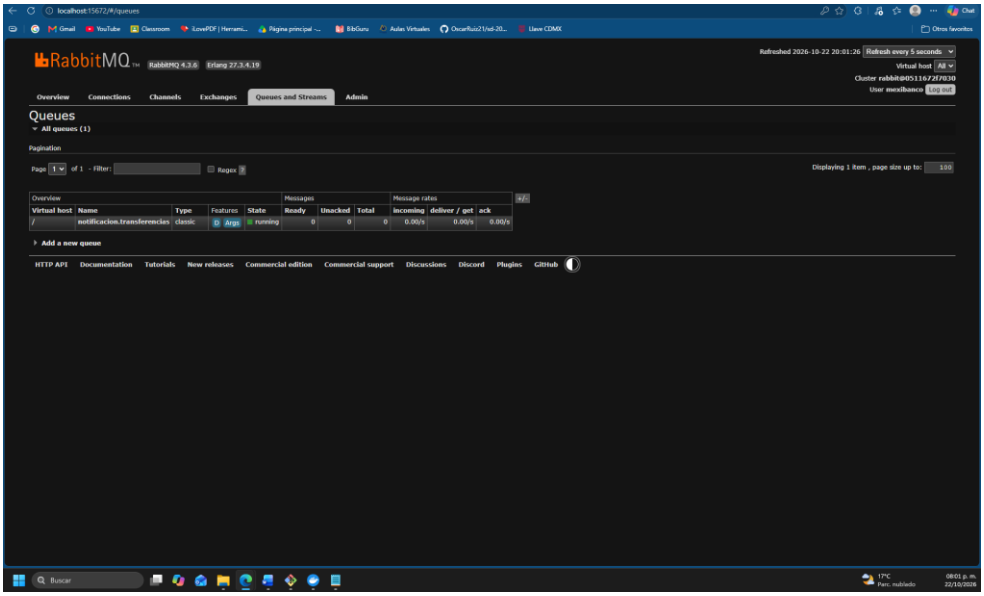
En conclusión, los publisher confirms ayudan a verificar que el broker aceptó la publicación, pero no garantizan el procesamiento de los eventos ni la atomicidad entre la base de datos y RabbitMQ. Para cubrir el segundo caso sería necesario un

mecanismo como el patrón Transactional Outbox, que permita registrar el evento de forma duradera junto con la transferencia y publicarlo posteriormente.

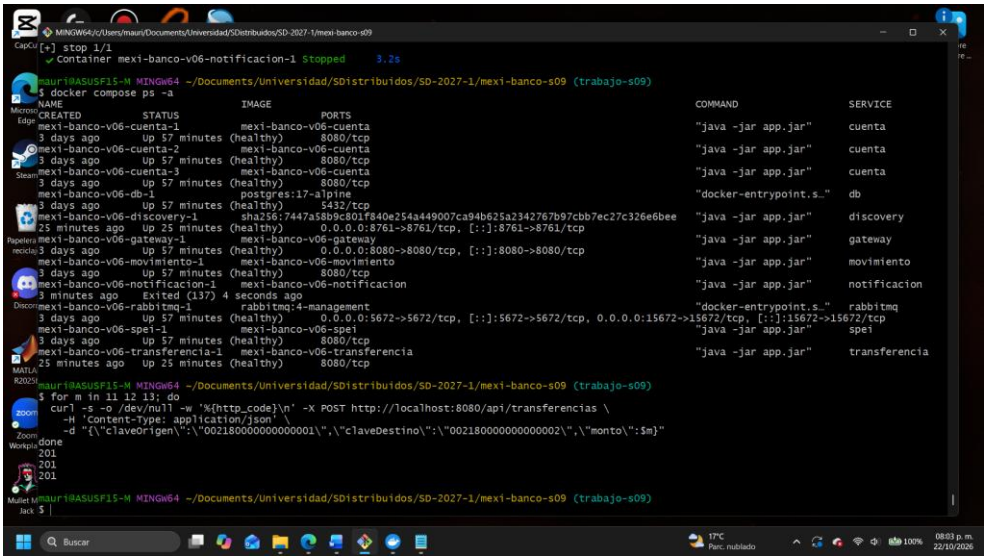
4.- Consumir el evento 22/10/2026. 7:53pm

4.5.- Pruébalo: el desacople en tiempo

Vemos notificacion.transferencias en el panel Queues and Streams.

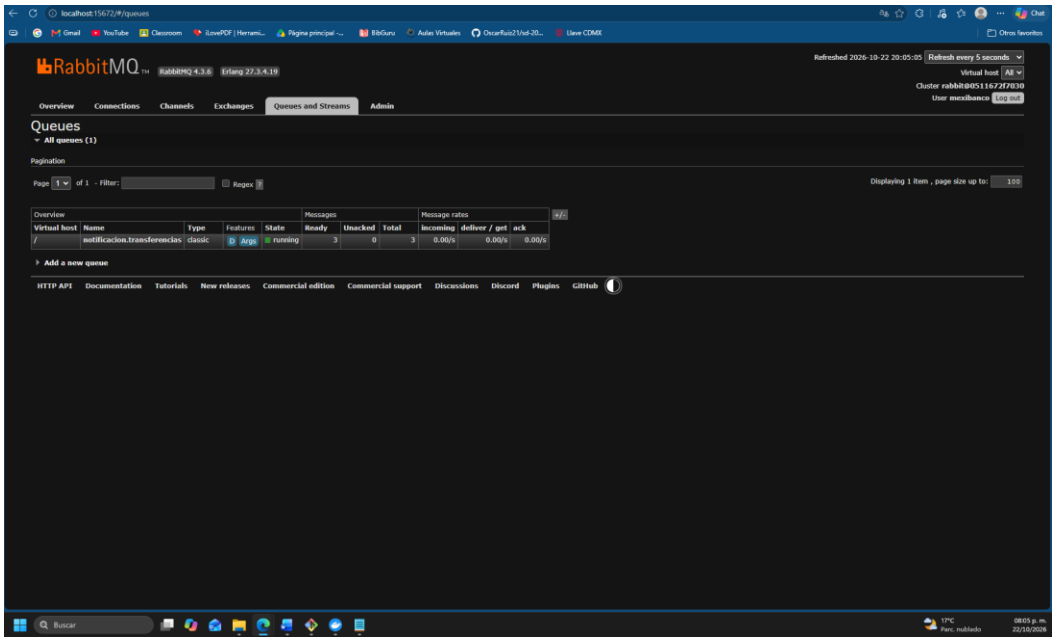


Ejecutamos las tres transferencias:

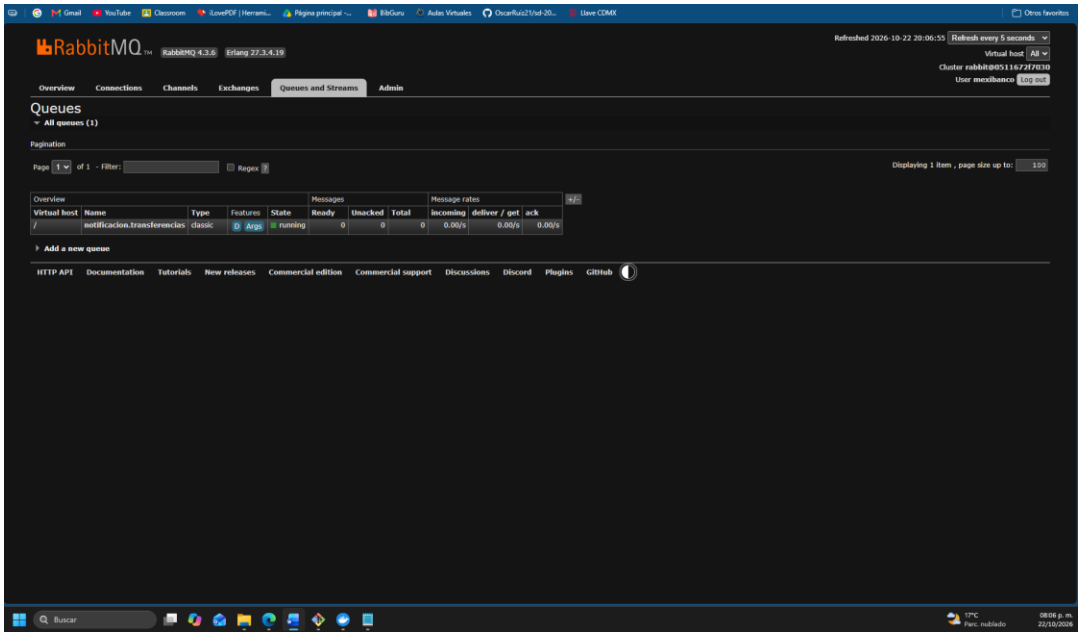


Consultamos los mensajes pendientes:

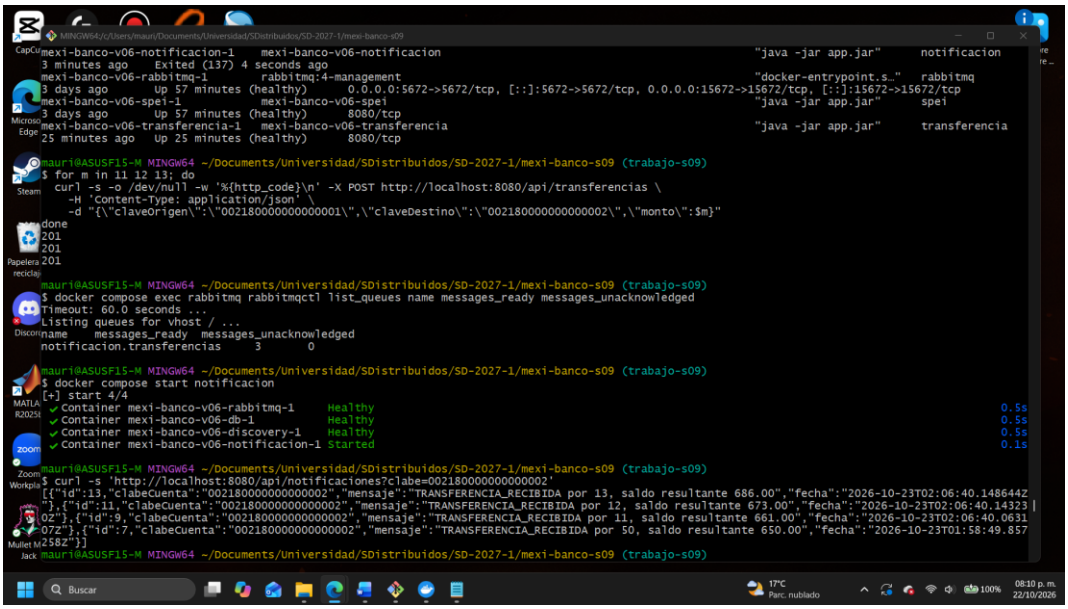
```
mauri@ASUSF15-M MINGW64 ~/Documents/Universidad/SDistribuidos/SD-2027-1/mexi-banco-s09 (trabajo-s09)
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     3       0
```



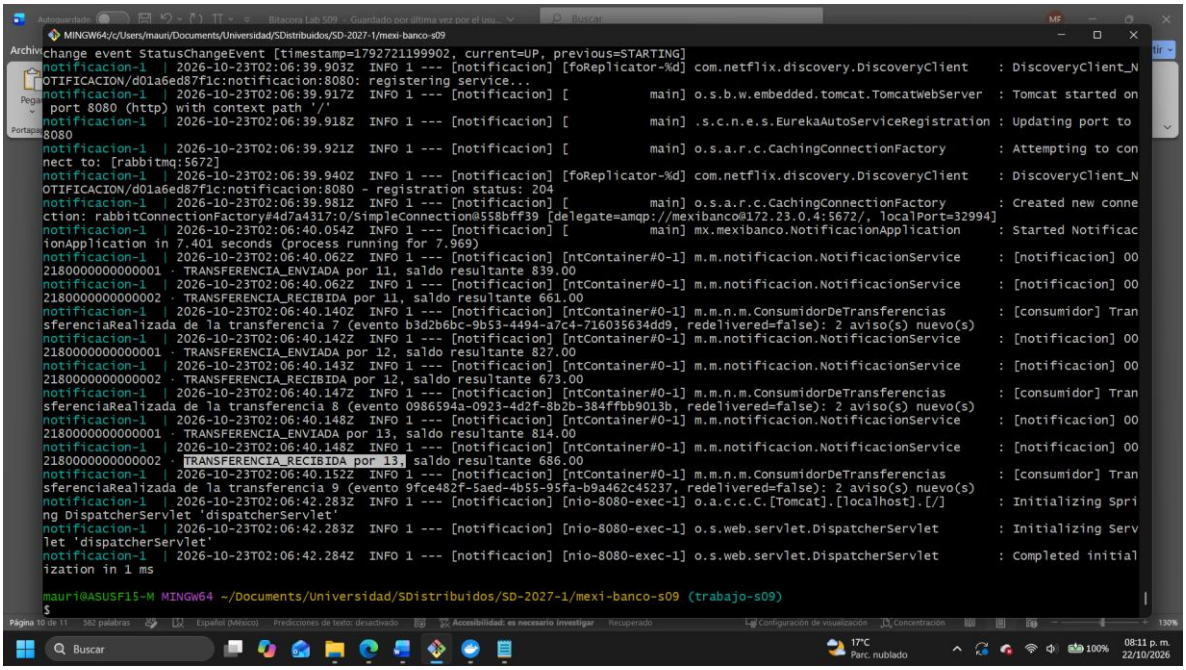
Despues de encender notificación:



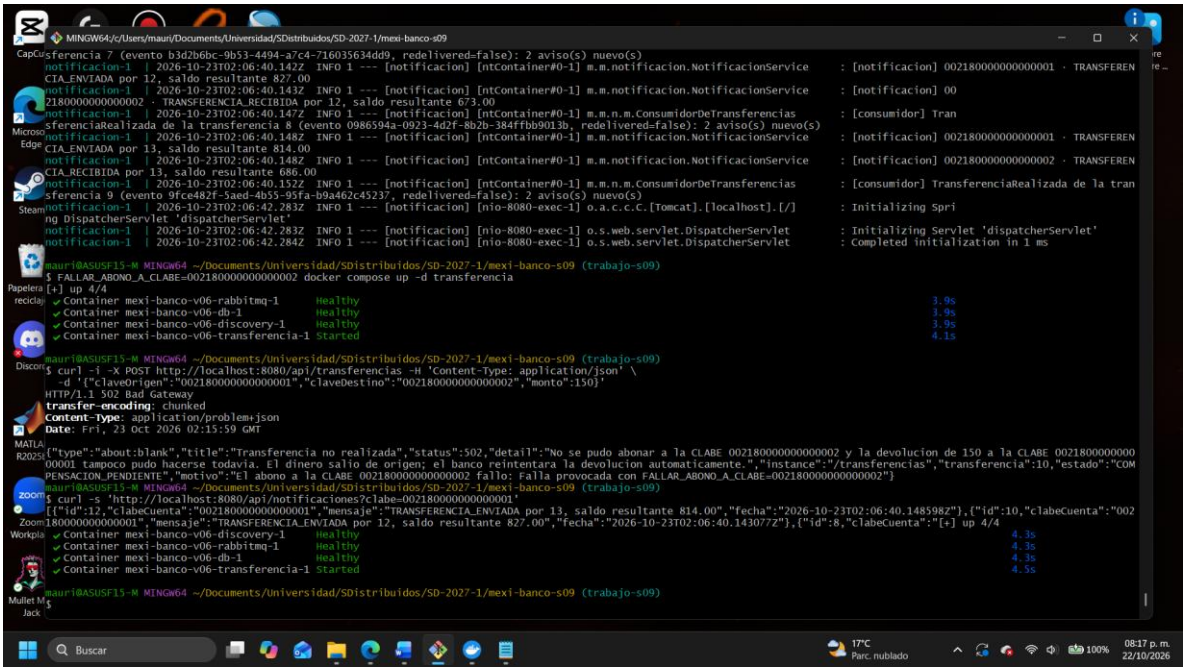
Consultamos la bandeja de Beto:



Consultamos los logs de Transferencia:



Probamos la revertida con la falla de la S08.



El ack se realiza después de guardar las notificaciones en la base de datos para evitar que se pierdan mensajes. Si el consumidor confirmara el mensaje antes de guardarlo y el servicio fallara entre ambas operaciones, RabbitMQ consideraría el mensaje procesado y lo eliminaría de la cola. Como consecuencia, las notificaciones nunca se guardarían y no habría forma de recuperarlas mediante la cola.

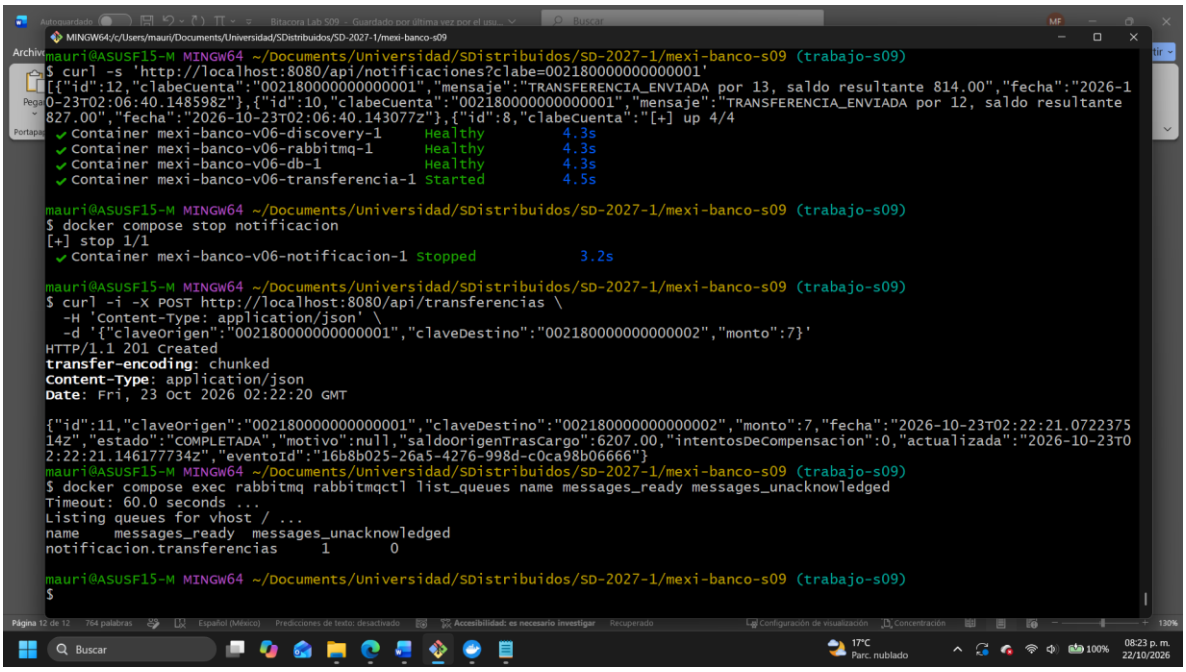
En cambio, al guardar primero y enviar el ack después, si el servicio falla entre ambas operaciones, RabbitMQ no recibe la confirmación y vuelve a entregar el mensaje cuando el consumidor se recupera. Esto permite procesarlo nuevamente, aunque existe el riesgo de que las notificaciones se guarden duplicadas si ya se habían guardado antes del fallo.

Por lo tanto, confirmar después de guardar evita perder mensajes, pero puede provocar duplicados. Para solucionar este último problema, se pueden implementar mecanismos de idempotencia que permitan reconocer los eventos ya procesados y evitar guardar las mismas notificaciones más de una vez.

5.- Tarea · El duplicado 22/10/2026 8:21pm

5.1.- Provócalo

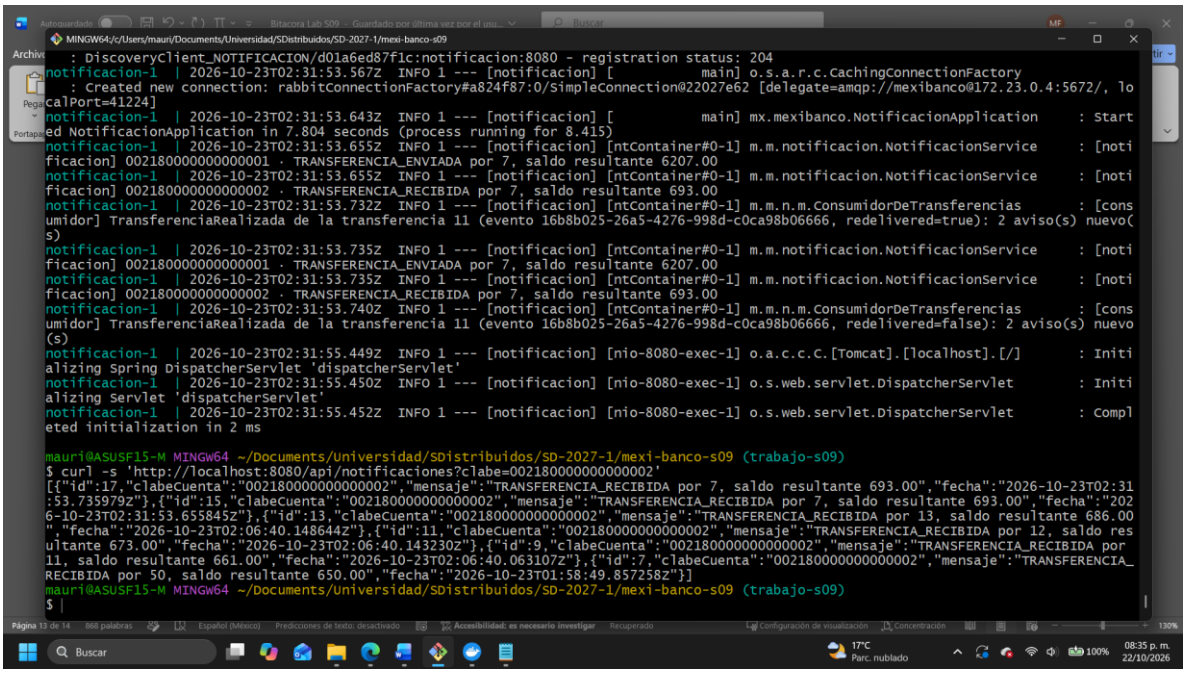
Detenemos notificación, generamos un evento nuevo y consultamos si se generó un mensaje pendiente.



Obtenemos el mensaje original en Queues and Streams utilizando la sección get messages. Para publicar el mismo evento otra vez, entramos en exchanges, transferencias y seleccionamos publish message con la routing key transferencia.realizada y con el JSON que obtuvimos en get messages como payload.

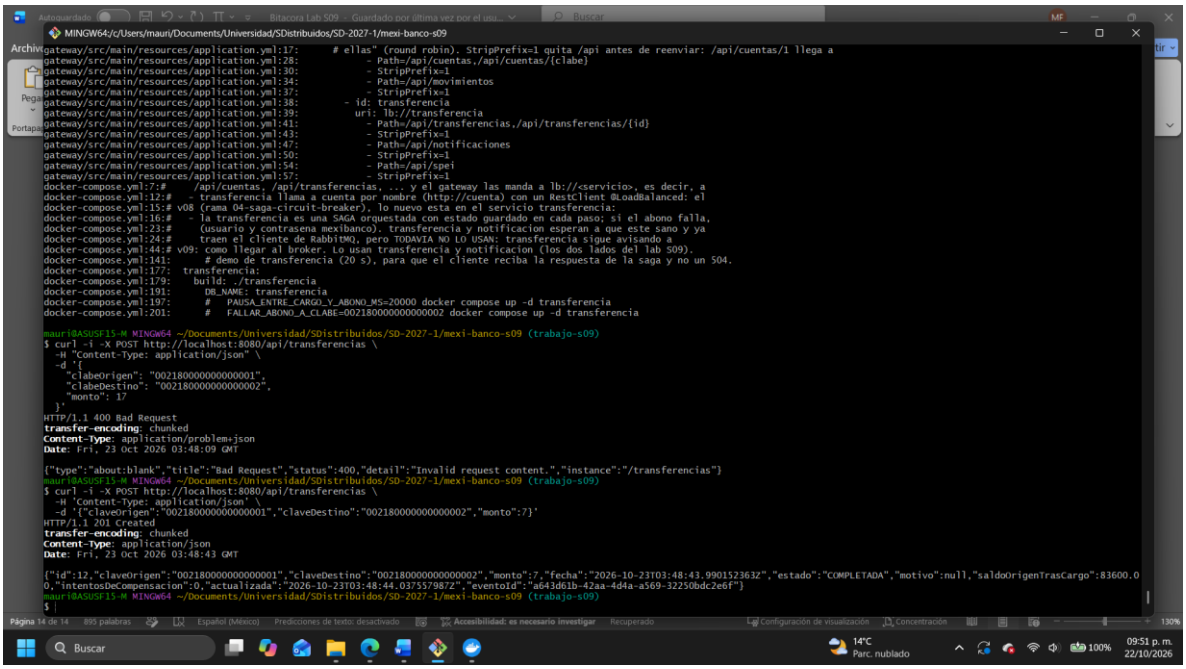
Vemos que en Queues and Streams tenemos dos mensajes en ready.

Consultando los logs de notificación vemos que se enviaron 2 notificaciones, y en la cuenta receptora encontramos dos notificaciones de TRANSFERENCIA_RECIBIDA.

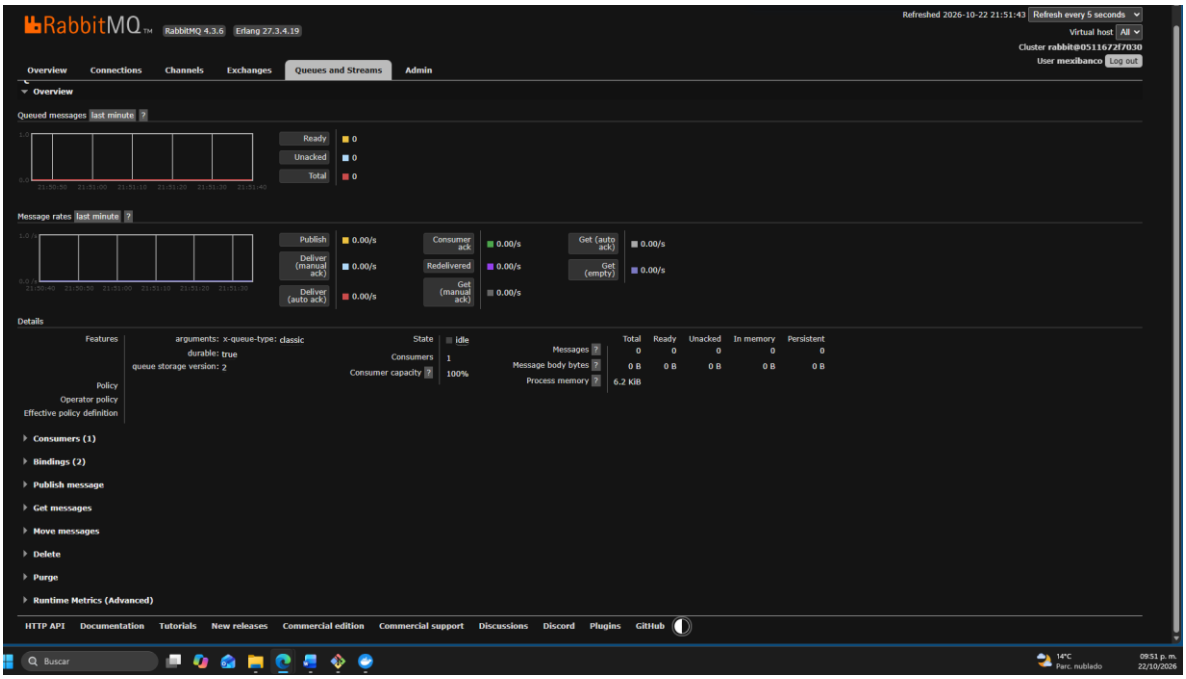


5.2.- Arréglalo

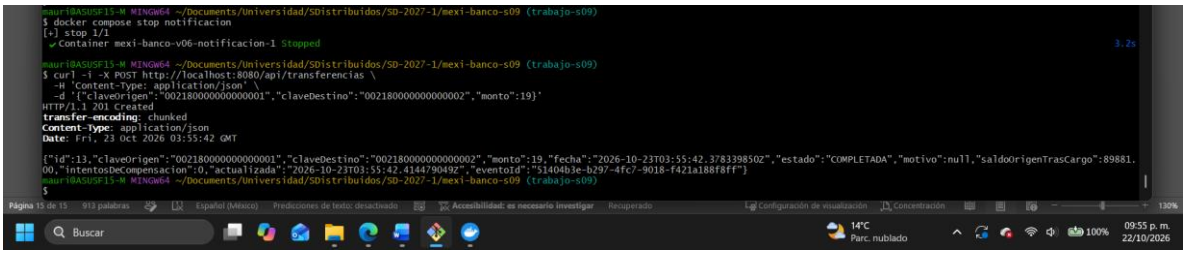
Creamos un nuevo mensaje igual al anterior.



En RabbitMQ notamos los valores de ready y unacked en 0.

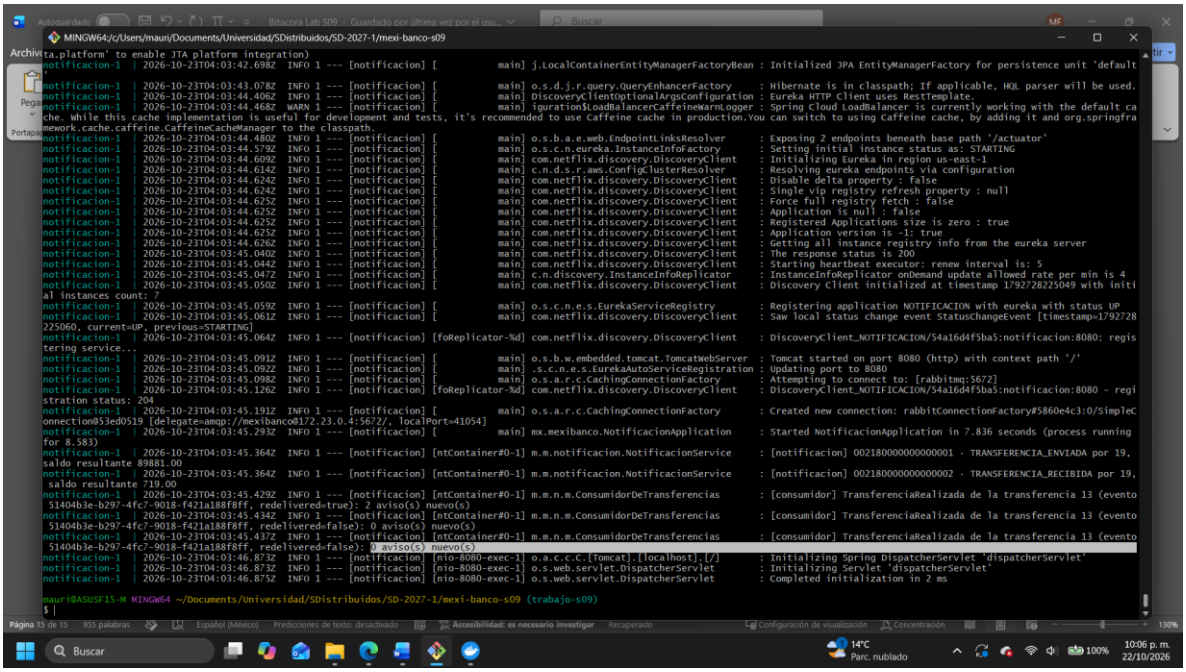


Detenemos notificación y creamos un vento nuevo, en este caso, con una transferencia de 19.



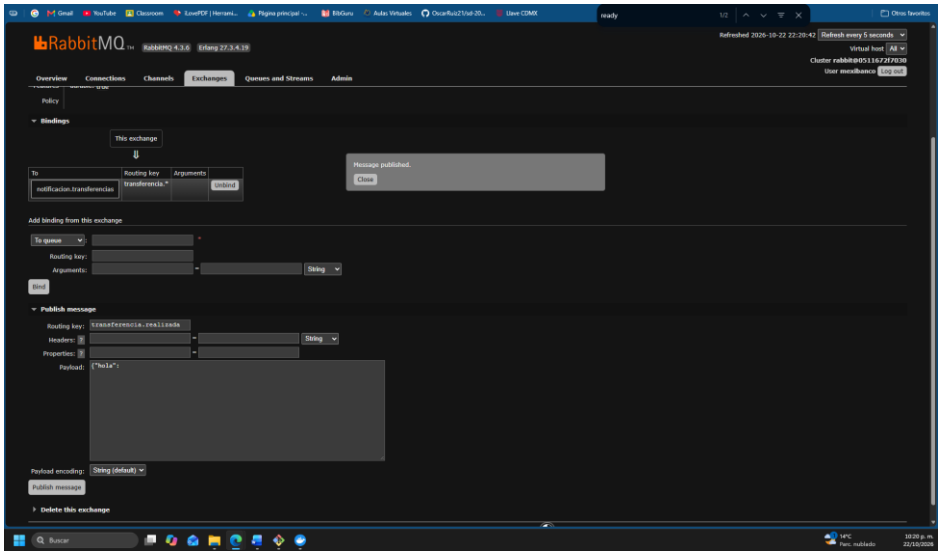
Obtenemos el json en la sección payload de get messages. En este caso, seleccionamos publish message dos veces. Revisando queues and streams tenemos 3 mensajes en ready.

Volvemos a encender notificación. Se reinician los contadores de Queues and Streams.
Consultamos los logs otra vez, vemos un solo juego de avisos con el identificador de 13, además de obtener 0 avisos nuevos en la entrega más reciente.

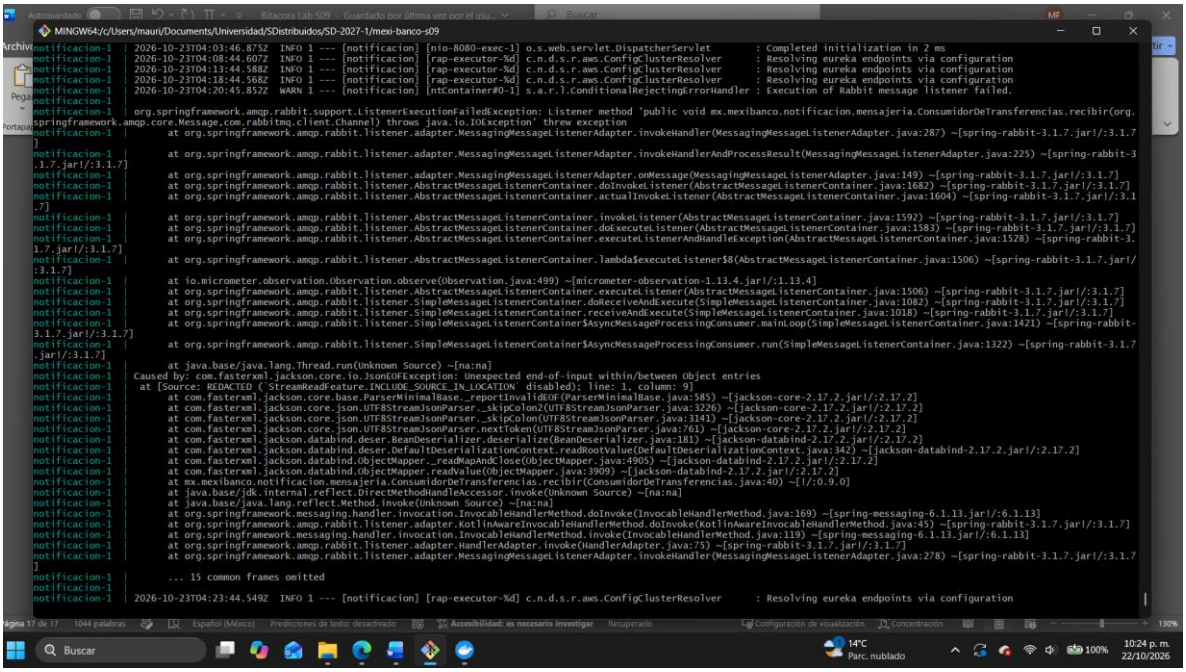


6.- Tarea : El mensaje envenenado 22/10/2026 – 10:16pm

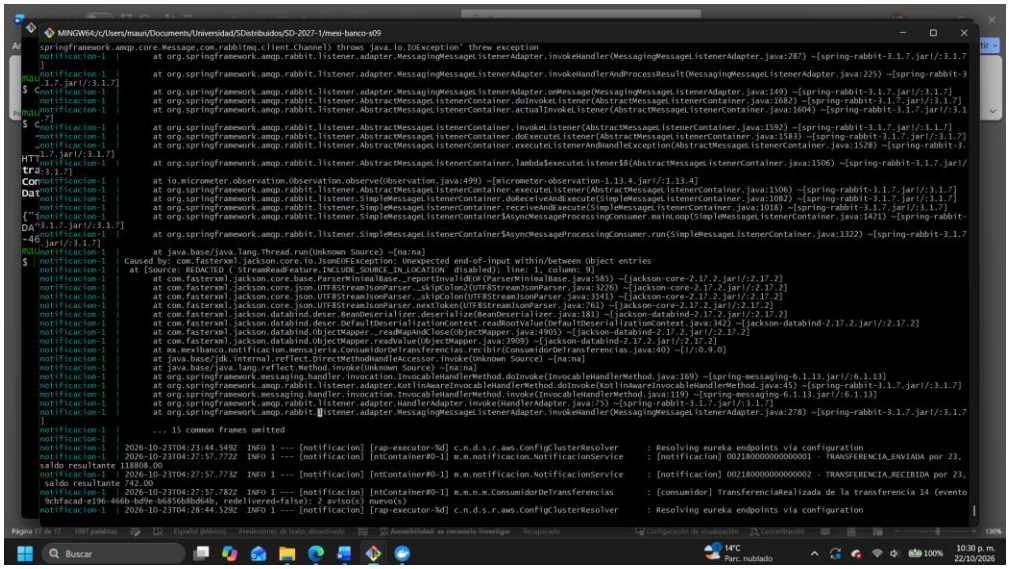
Usando el comando “docker compose logs -f notificacion” dejamos corriendo los logs de notificación. En RabbitMQ, nois vamos a exchanges, transferencias, publish message. Usamos transferencia.realizada como Routing key. En Payload, escribimos “hola”. Damos clic en publish message.



RabbitMQ muestra un mensaje Unacked. El log nos confirma el error JsonEOFException, siendo que se toma al mensaje como incompleto y no se puede convertir en un evento.

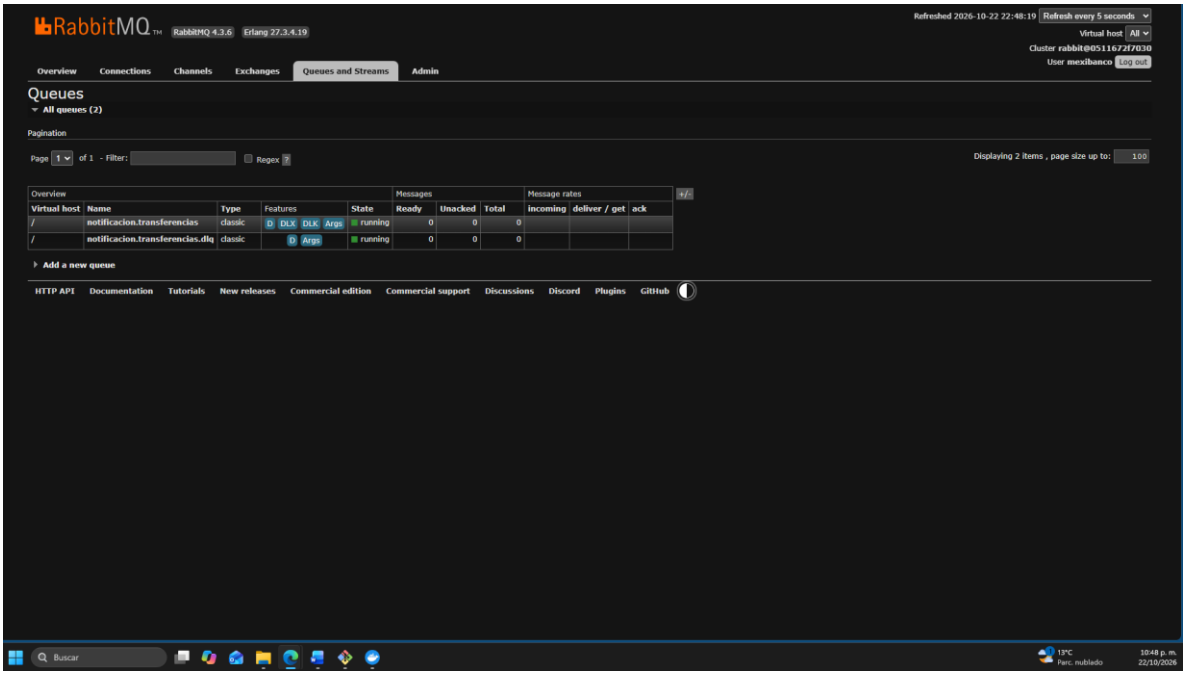


Abrimos una segunda terminal de Git Bash, en la raíz de mexi-banco-s09, y ejecutamos una transferencia normal de $23. Acaba con la salida 201 (created) y genera el evento 9cbfacad-e196-466b-bd9e-b6856b8bd64b.
Vemos que si se tomó en cuenta esta transferencia en los logs.

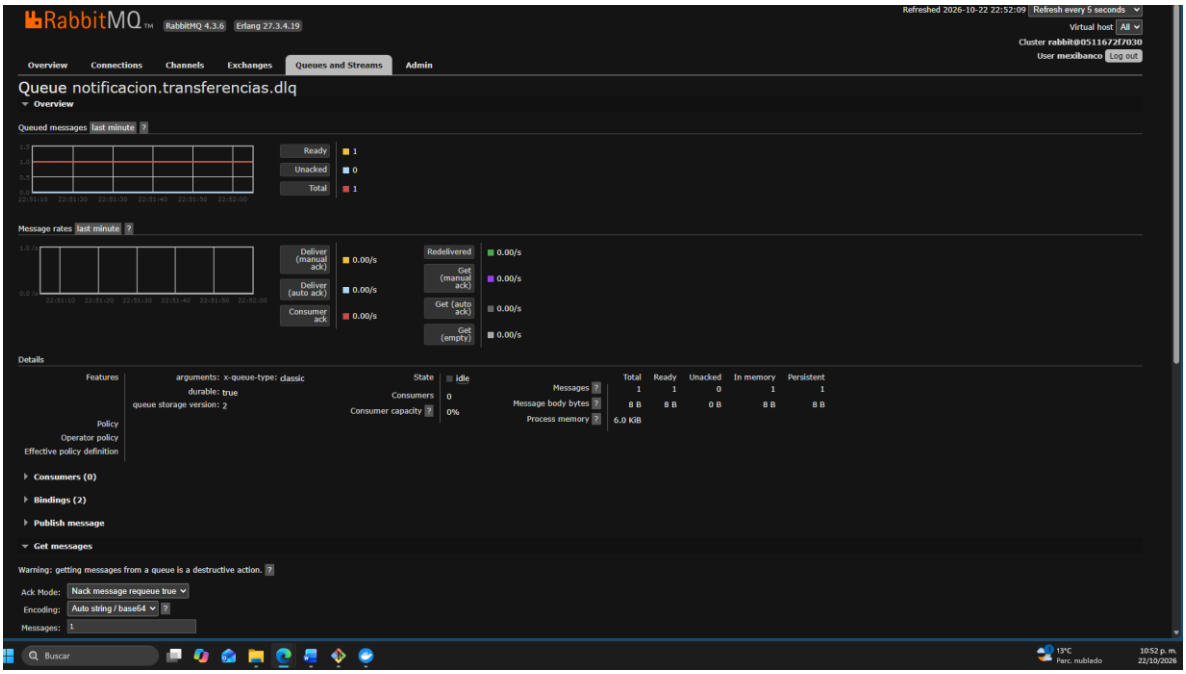


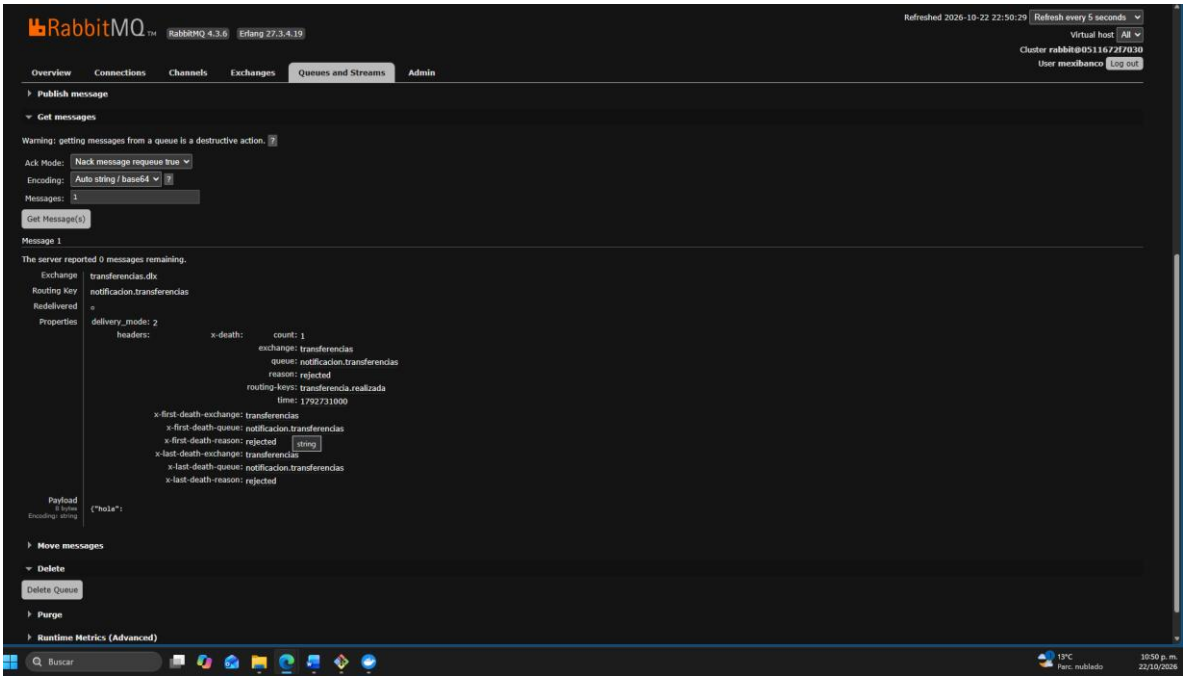
Ahora, detenemos notificación y eliminamos la cola en RabbitMq para descartar el mensaje envenenado.

Volvemos a construir notificación. En RabbitMQ observamos la creación de la cola.dlq, siendo la cola para mensajes fallidos. En Exchanges, también existe transferencias.dlx.



Volvemos a mandar el mensaje envenenado. Ahora, este aparece en la cla .dlq al dar clic en get messages, con el encabezado x-death y razón rejected.





Preguntas de cierre

1. Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el publisher confirm?

Si RabbitMQ está caído, la transferencia puede completarse, pero el evento no se puede publicar y el aviso podría perderse. El publisher confirm no lo soluciona porque

solo confirma si RabbitMQ recibió el mensaje, por lo que no puede garantizar su publicación cuando el servidor está caído ni recuperar automáticamente el evento perdido.

2. ¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?

El ack va después de guardar la notificación para evitar confirmar un mensaje antes de procesarlo. Si se confirma antes de guardar y ocurre un error, el mensaje podría perderse. Si se confirma después, RabbitMQ puede volver a entregarlo si el consumidor falla antes de confirmar, aunque esto puede generar duplicados, por lo que usamos idempotencia para evitarlos.

3. Elige y di qué sacrificaste: al quitar la llamada síncrona ganaste que el aviso no tumbe la transferencia. ¿Qué perdiste?

- Elección: usar mensajería asíncrona con RabbitMQ.
- Porque: la transferencia puede completarse sin esperar a que el servicio de notificaciones procese el evento.
- Sacrifico: inmediatez y garantía de que el aviso se genere. Si falla la publicación, el evento puede perderse porque no implementamos un mecanismo de recuperación como un outbox transaccional.